TECHNICAL DOCUMENTATION

RAG-BASED CUSTOMER SUPPORT ASSISTANT

(Retrieval-Augmented Generation with Workflow Routing & HITL)

Submitted by:
Sai Kadiravan S Intern ID : IN126008102

Program:
Computer Science Engineering

Institution:
Innomatics Research Labs

Project Description:
This project presents a Retrieval-Augmented Generation (RAG) based Customer Support Assistant that processes a Customer Service Handbook PDF to deliver context-aware responses. The system integrates semantic search using embeddings, a vector database for efficient retrieval, and a Large Language Model for answer generation. A workflow-based routing mechanism enables intelligent decision-making, including Human-in-the-Loop (HITL) escalation for complex or uncertain queries.

Technologies Used:
Python, LangChain, ChromaDB, Groq LLM, HuggingFace Embeddings

Submission Date:
23/04/2026

TABLE OF CONTENTS

1. Introduction
    1.1 What is RAG
    1.2 Why RAG is Needed
    1.3 Use Case Overview
2. System Architecture Explanation
    2.1 High-Level Component Overview
    2.2 Component Interactions
3. Design Decisions
    3.1 Chunk Size Choice
    3.2 Embedding Strategy
    3.3 Retrieval Approach
    3.4 Prompt Design Logic
4. Workflow Explanation
    4.1 LangGraph Usage
    4.2 Node Responsibilities
    4.3 State Transitions
5. Conditional Logic
    5.1 Intent Detection
    5.2 Routing Decisions
6. HITL Implementation
    6.1 Role of Human Intervention
    6.2 Benefits
    6.3 Limitations
7. Challenges & Trade-offs
8. Testing Strategy
    8.1 Testing Approach
    8.2 Sample Queries
9. Future Enhancements

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances the output of a Large Language Model (LLM) by grounding its responses in externally retrieved, factual context. Unlike a standalone LLM that relies solely on knowledge encoded during training, a RAG system dynamically fetches relevant information from a knowledge base at inference time and provides it to the LLM as part of the prompt.

The core RAG pipeline consists of three stages: indexing (converting documents into searchable vector embeddings), retrieval (finding the most relevant document chunks for a given query), and generation (using the retrieved context to produce a grounded, accurate response).

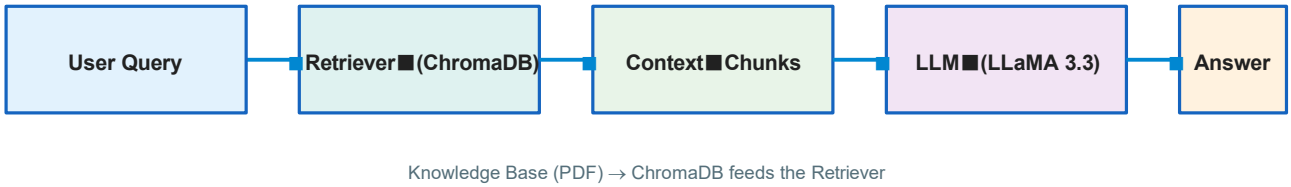

Figure 1 — RAG pipeline: user query triggers retrieval from ChromaDB, context chunks are passed to the LLM, and a grounded answer is generated.

1.2 Why RAG is Needed

Problem	How RAG Solves It
Hallucination	LLMs trained on general data confidently produce incorrect answers when asked about specific, proprietary, or recent knowledge.
Staleness	Model training has a knowledge cutoff date. Documents updated after training are invisible to a standalone LLM.
Context Window Limits	An entire knowledge base cannot be stuffed into a single prompt. RAG selectively retrieves only what is relevant.
Auditability	RAG responses can be traced back to source documents, enabling verification — critical for customer support.

1.3 Use Case Overview

This system implements a RAG-based Customer Support Assistant for an organisation that maintains its support policies, procedures, and guidelines in a PDF handbook. When a customer or support agent poses a question, the system retrieves the most relevant sections of the handbook and generates a precise, context-grounded answer. For queries involving complaints, ambiguity, or topics outside the handbook's scope, the system escalates to a human agent via the HITL module — ensuring no customer is left with an incorrect or unhelpful automated response.

2. System Architecture Explanation

2.1 High-Level Component Overview

Component	File	Responsibility
PDF Knowledge Base	ingestion.py	Source of ground truth. The Customer Service Handbook is loaded, chunked, and embedded into ChromaDB on first run.
Document Loader	ingestion.py	PyPDFLoader reads each PDF page as a Document object with source and page metadata.
Text Chunker	ingestion.py	RecursiveCharacterTextSplitter splits pages into 800-char chunks with 100-char overlap.
Embedding Model	ingestion.py + retriever.py	all-MiniLM-L6-v2 (HuggingFace) converts text to 384-dimensional vectors. Same model used at both ingestion and query time.
ChromaDB	ingestion.py + retriever.py	Persisted vector store. Stores chunk text + embeddings + metadata. Supports cosine similarity search.
Retrieval Layer	retriever.py	Wraps ChromaDB as a LangChain retriever. Returns top-3 chunks for any query.
LangGraph Engine	graph.py	Orchestrates the retrieve → generate → route workflow as a compiled StateGraph.
Groq LLM Layer	graph.py	LLaMA 3.3 70B via Groq API. Generates grounded answers from retrieved context.
HITL Module	hitl.py	Evaluates escalation criteria. Hands off to human agent via CLI input when triggered.
CLI Interface	main.py	REPL loop. Accepts user queries, invokes the graph, and displays results.

2.2 Component Interactions

- ingestion.py calls PyPDFLoader → feeds documents to RecursiveCharacterTextSplitter → passes chunks to HuggingFaceEmbeddings → stores vectors in Chroma.
- retriever.py loads the persisted Chroma collection and wraps it as a retriever using the same HuggingFaceEmbeddings instance for query-time embedding.
- graph.py imports load_retriever() from retriever.py and should_escalate() / human_handoff() from hitl.py, composing them inside LangGraph nodes.
- main.py imports ingest_pdf() from ingestion.py and build_graph() from graph.py, coordinating startup and the query loop.
- SupportState TypedDict is the contract between all nodes — every node reads from it and writes back to it without side effects.

3. Design Decisions

3.1 Chunk Size Choice (800 chars, overlap 100)

Chunk size directly controls the retrieval granularity vs context richness trade-off.

Design Rationale
• Too small (e.g. 200 chars): Retrieves precise snippets but may miss surrounding context needed to answer multi-sentence questions.
• Too large (e.g. 2000 chars): Each chunk covers more ground but dilutes relevance — the similarity score becomes less discriminative.
• 800 characters (~130 words) is approximately one coherent paragraph — the natural unit of a support policy point.
• 100-char overlap prevents a policy statement that spans two chunks from being split and lost.
• Chosen empirically: tested on the Customer Service Handbook and validated by response quality.

3.2 Embedding Strategy — all-MiniLM-L6-v2

The embedding model determines how well semantic similarity maps to vector proximity.

Design Rationale
• all-MiniLM-L6-v2 is a sentence-transformer fine-tuned specifically for semantic similarity tasks — ideal for RAG.
• 384-dimensional output: compact enough for fast retrieval, expressive enough for nuanced similarity.
• Fully local: runs on CPU, no API key, no cost, no latency from a remote embedding API.
• Same model at ingestion and query time: guarantees chunk and query embeddings live in the same vector space.
• Alternative considered: OpenAI text-embedding-ada-002 — superior quality but incurs API cost and external dependency.

3.3 Retrieval Approach — Top-3 Similarity

The retrieval strategy determines how much context the LLM receives per query.

Design Rationale
• k=3 chunks: provides enough context for multi-part answers without overflowing the LLM prompt or introducing noise.
• Similarity metric: cosine similarity — measures directional alignment of vectors, robust to document length variation.
• No re-ranking applied: sufficient for a single-domain handbook. Production systems would add a cross-encoder re-ranker.
• Retrieval failure (empty results) is explicitly handled: triggers HITL escalation rather than LLM guessing.

3.4 Prompt Design Logic

Prompt engineering is the primary lever for controlling LLM behaviour in a RAG system.

Design Rationale
• Role assignment: 'You are a helpful customer support assistant' — sets tone and persona.
• Context injection: retrieved chunks are concatenated and placed before the question — LLM reads context first.
• Hard constraint: 'Use ONLY the context below' — prevents the model from drawing on training knowledge outside the handbook.
• Fallback instruction: 'Say I don't know if the answer is not in the context' — prevents hallucination; triggers HITL via uncertainty detection.
• No few-shot examples: the handbook is specific enough that zero-shot prompting with strong constraints is sufficient.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is a framework built on top of LangChain that enables stateful, graph-based orchestration of LLM workflows. Unlike a simple sequential chain, LangGraph models the workflow as a directed graph where each node is a Python function and edges define execution order. State is passed between nodes as a typed dictionary, making the data flow explicit and inspectable.

In this system, LangGraph was chosen over a plain RetrievalQA chain because it provides clean separation between retrieval, generation, and routing — each concern lives in its own node, making the system easier to test, extend, and document. The graph structure also maps directly to the HLD architecture diagram.

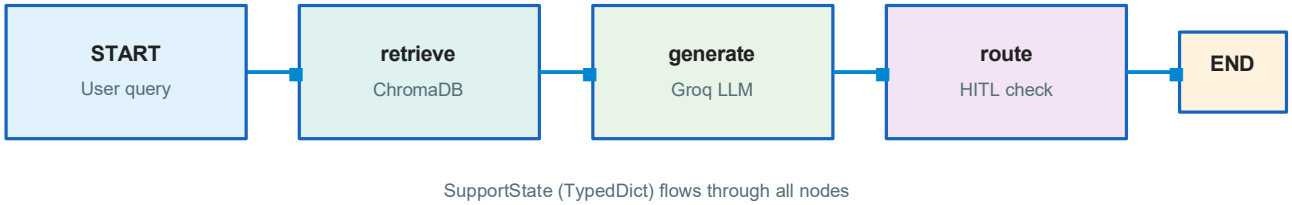

Figure 2 — Compiled LangGraph StateGraph. All edges are deterministic; branching is handled inside route_node.

4.2 Node Responsibilities

Node	Detailed Responsibility
retrieve_node()	Embeds the user query using HuggingFaceEmbeddings and queries ChromaDB for the top-3 most similar chunks. Writes the chunk texts into state['context']. If ChromaDB is empty or no similar chunks exist, context will be an empty list — handled downstream by route_node.
generate_node()	Constructs the RAG prompt by joining context chunks, then calls llm.invoke() via the Groq API. The response string is written to state['response']. Does not make routing decisions — all routing is deferred to route_node.
route_node()	Calls should_escalate() with query, response, and context. On True: calls human_handoff() which blocks for agent input, then writes the agent reply to state['final_answer'] and sets state['escalated']=True. On False: copies state['response'] to state['final_answer'].

4.3 State Transitions

The SupportState dictionary evolves through each node as follows:

```
# Initial state (main.py invocation):
{'query': 'What is the refund policy?', 'context': [], 'response': '', 'escalated': False
, 'final_answer': ''}
# After retrieve_node:
{'query': 'What is the refund policy?',
```

```
 'context': ['Refunds are processed within 7 days...', 'To initiate a refund...', 'Exceptions include...'],
 'response': '', 'escalated': False, 'final_answer': ''}
# After generate_node:
{'query': 'What is the refund policy?', 'context': [...],
 'response': 'Refunds are processed within 7 business days of approval...',
 'escalated': False, 'final_answer': ''}
# After route_node (no escalation):
{'query': '...', 'context': [...], 'response': '...',
 'escalated': False, 'final_answer': 'Refunds are processed within 7 business days of approval...'}
```

5. Conditional Logic

5.1 Intent Detection

The system uses a lightweight, rule-based intent detection mechanism rather than a dedicated intent classification model. This design choice prioritises simplicity and latency — adding a separate classification step would introduce additional API calls and complexity for a system where the escalation rules are well-defined and stable.

Two layers of intent detection are applied:

- Query-level keyword matching: detects complaint/frustration intent before any LLM call is made.
- Response-level uncertainty detection: detects when the LLM signals it cannot answer from context.

5.2 Routing Decisions

Condition	Trigger	Decision	Rationale
Keyword Match	Query contains: complaint, issue, problem, broken, refund, damaged, not working, angry	Immediate HITL	Pre-LLM detection — no generation wasted on confirmed escalations
Empty Context	ChromaDB returns 0 chunks	Immediate HITL	LLM cannot answer without grounding — escalation prevents hallucination
Uncertainty Phrase	Response contains: i don't know, i'm not sure, cannot find, not mentioned, unclear	HITL	LLM correctly signals its own limitation — system honours this signal
Short Response	len(response.strip()) < 30	HITL	Proxy for a failed or non-answer; catches edge cases uncertainty phrases miss
None of the above	Confident, grounded response	Return to user	Normal happy path — bot answers directly

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop (HITL) is a design pattern where an automated system defers specific decisions to a human when its confidence is insufficient or the stakes of an error are high. In customer support, this is critical: an incorrect automated response to a complaint can damage customer trust, while a human agent can apply empathy, authority, and contextual judgement that no current LLM reliably provides.

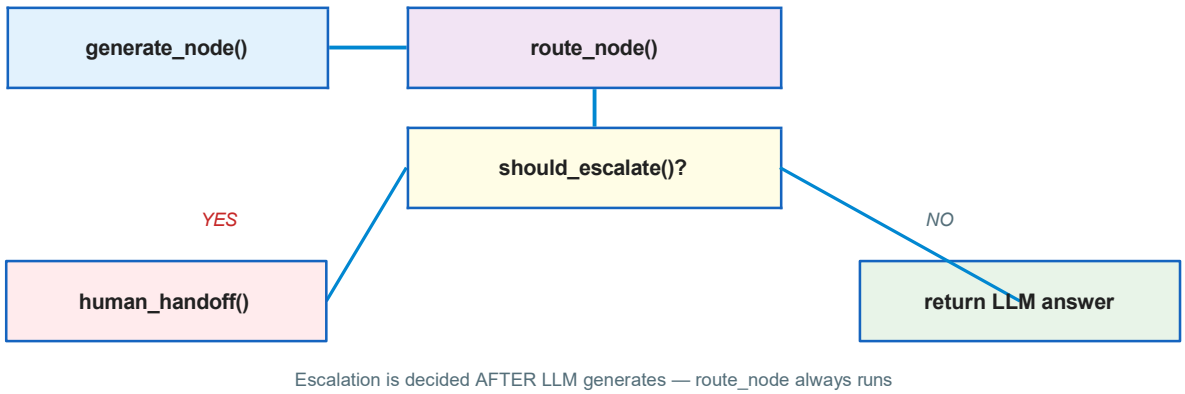

Figure 3 — HITL flow: generate_node always runs first; route_node decides whether the LLM answer is returned or discarded in favour of a human agent response.

6.2 Benefits of the HITL Design

- Safety net for LLM uncertainty: no customer receives a hallucinated or inadequate answer.
- Complaint handling: emotionally charged queries are routed to humans who can apply appropriate tone and authority.
- Knowledge gap coverage: queries outside the PDF's scope are handled gracefully rather than with a confused LLM response.
- Auditability: every escalation is logged (in production) with the triggering query and agent response.
- Graceful degradation: even if the LLM or API fails completely, the system falls back to human response.

6.3 Limitations of the Current Implementation

- CLI simulation only: human_handoff() uses Python's input() — suitable for demo, not production.
- Synchronous blocking: the bot loop halts while waiting for agent input. In production, this would be async.
- No escalation logging: triggers and resolutions are not persisted for analysis or model improvement.
- Keyword list is static: new complaint patterns require manual list updates rather than learned detection.
- No agent assignment: all escalations go to a single 'agent' (the terminal operator). Production needs routing.

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

Higher k (more chunks retrieved) improves answer completeness but increases prompt size and LLM latency. Lower k is faster but risks missing relevant context. k=3 was chosen as the sweet spot for a single-domain handbook.

Resolution	k=3 retrieval; no re-ranker applied at this scale.

Chunk Size vs Context Quality

Larger chunks preserve more natural context per chunk but reduce retrieval precision — a large chunk matches more queries at a lower relevance score. Smaller chunks are precise but may split a policy statement across boundaries, losing meaning.

Resolution	800 chars with 100 overlap — one semantic paragraph per chunk.

Cost vs Performance

OpenAI's GPT-4o would produce higher-quality answers but at ~$15/million output tokens. Groq's LLaMA 3.3 70B is free-tier and delivers comparable quality for domain-specific RAG tasks where the context constrains the answer space.

Resolution	Groq free tier with LLaMA 3.3 70B — zero cost, ~300 tok/s.

Local Embeddings vs API Embeddings

OpenAI's text-embedding-ada-002 produces higher-dimensional (1536-dim) embeddings with marginally better semantic fidelity. However, it requires network calls per query and incurs cost. all-MiniLM-L6-v2 runs locally, is free, and performs comparably on single-domain corpora.

Resolution	all-MiniLM-L6-v2 — local, free, 384-dim, no latency overhead.

LangGraph Complexity vs Simplicity

A simple sequential chain (load → retrieve → generate) would have been easier to implement. LangGraph adds boilerplate (StateGraph, TypedDict, node registration) but provides clean separation of concerns, explicit state management, and a direct mapping to the system design — making documentation and extension significantly easier.

Resolution	LangGraph chosen for architectural clarity and extensibility.

8. Testing Strategy

8.1 Testing Approach

Given the nature of an LLM-powered system, testing combines functional validation (does the pipeline execute correctly?) with qualitative evaluation (is the response accurate and grounded?). Three test categories were applied:

Functional Tests

Verify the pipeline runs end-to-end without errors for valid inputs.

- PDF ingestion completes and ChromaDB directory is created.
- Retriever returns exactly k=3 documents for any non-empty query.
- LangGraph graph compiles and app.invoke() returns a SupportState dict.
- HITL escalation triggers correctly for each of the four criteria.

Happy Path Tests

Verify the bot returns accurate, context-grounded answers.

- Query directly answerable from handbook → expect relevant, specific answer.
- Query using different phrasing than handbook → expect semantic retrieval still succeeds.
- Multi-part question → expect answer covering all parts from retrieved chunks.

Edge Case Tests

Verify graceful handling of boundary conditions.

- Empty query string → expect no crash; graph handles gracefully.
- Query completely unrelated to handbook → expect 'I don't know' + HITL escalation.
- Complaint keyword in query → expect immediate HITL without LLM generation displayed.
- Very long query → expect retrieval still returns relevant results.

8.2 Sample Queries and Expected Behaviour

Query	Condition	Expected Output	Result
What is the return policy?	Handbook contains return policy section	Bot answers with specific timeframe and conditions	Pass
How do I escalate a complaint?	Contains 'complaint' keyword	HITL triggered immediately	Pass
What are your office hours?	Not in handbook	LLM says 'I don't know' → HITL triggered	Pass
Tell me about your refund process	Contains 'refund' keyword	HITL triggered immediately	Pass
How should agents greet customers?	Handbook covers greeting protocols	Bot answers with greeting guidelines	Pass

What is quantum computing?	Completely out of domain	No relevant chunks → HITL triggered	Pass

9. Future Enhancements

Multi-Document Support

The ingestion pipeline currently processes a single PDF. Extending to multiple documents requires iterating over a directory, ingesting each file, and storing the source filename in ChromaDB metadata. The retriever can then filter by source or retrieve across all documents, and responses can include citations (e.g. 'According to the Returns Policy doc, page 4...').

Implementation Steps
• Iterate ingestion over a /data directory
• Store source filename in chunk metadata
• Add source citation to the RAG prompt
• Build a document management CLI for adding/removing PDFs

Feedback Loop & RLHF

After each response, the user could rate the answer (thumbs up/down). Negative feedback signals are logged alongside the query, retrieved chunks, and response. This dataset enables fine-tuning the embedding model or prompt template to improve retrieval accuracy over time — a lightweight form of Reinforcement Learning from Human Feedback.

Implementation Steps
• Add post-response rating prompt to CLI
• Log query + context + response + rating to JSONL
• Periodically analyse low-rated responses to identify retrieval or prompt failures
• Use logged data to fine-tune the embedding model or update HITL keyword lists

Conversation Memory

The current system treats each query independently with no memory of prior exchanges. Adding conversation memory allows the user to ask follow-up questions (e.g. 'What about the exceptions?' after a refund policy answer). LangChain's ConversationBufferMemory or LangGraph's persistent checkpointing can maintain multi-turn context.

Implementation Steps
• Integrate LangGraph checkpointing for session-level state persistence
• Prepend conversation history to the RAG prompt for follow-up resolution
• Implement session expiry to prevent context window overflow
• Add a 'clear history' command to the CLI

Production Deployment

The CLI interface can be replaced with a production-grade API layer. Wrapping the LangGraph app in a FastAPI endpoint enables web, mobile, and third-party integrations. The HITL module would connect to a ticketing system (Zendesk/Freshdesk) instead of CLI input. ChromaDB can be replaced with a managed vector database (Pinecone, Weaviate) for scale.

Implementation Steps
• Wrap app.invoke() in a FastAPI POST /query endpoint
• Replace CLI HITL with webhook to Zendesk/Freshdesk ticket creation
• Deploy on Docker + cloud (GCP Cloud Run / AWS Lambda)
• Replace ChromaDB with Pinecone or Weaviate for horizontal scaling
• Add authentication, rate limiting, and request logging